How familiar are you with engineering open house (EOH)?

clicker.cs.illinois.edu

Q1

~Code~

101

CS 340

AIOHTTP

Updates

1. MP6 graded - Report an error for a regrade request before April 15th
    a. If the files you submitted before the deadline work locally, show **Prof. Schatz** in office hours before **April 15th** for full credit back.
2. MP 8 - aiohttp lite, due next week
    a. Double check the files you submit
3. HW 7 out

Upcoming Schedule

next week

	Tues	thurs
mp9	APIs	Websockets
MP10	AI	authentication
final project	review	exam 3 → Guest speaker

X no class art change

May 11th

check-off

aiohttp

Today's LGs - Be able to make simple servers with aiohttp

Be able to complete MP 8

- Decorators
- Async
- ect...

Agenda

1. Python functions
2. Async
3. AIOHTTP

Functions - first-class objects

```
def greet():
    print("hi")

f = greet
f()
```

Functions - first-class objects

```
def greet():
    print("hi")
```

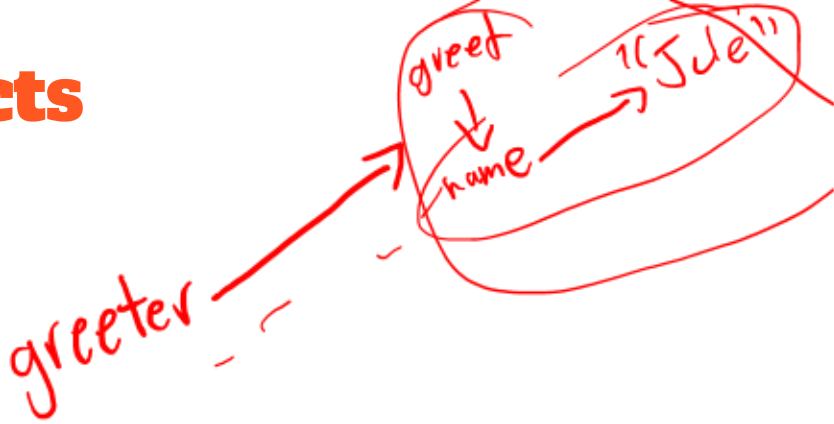
```
def run(func):
    func()

run(greet)
```

Functions - first-class objects

```
def make_greeter(name):
    def greet():
        print(f"Hello {name}!")
    return greet

greeter = make_greeter("Jule")
greeter()
```



What prints?

```
def add_one(x):
    return x + 1

def run(f, value):
    return f(value)

print(run(add_one, 4))
```

5

5

4

5

What prints?

```
x = 4
def thing1():
    x = 20
    return x

def thing2():
    x = -20
    return x

if x > 0:
    winner = thing1()
else:
    winner = thing2()
print(winner)
```

=4

20

winner

20

clicker.cs.illinois.edu

Q2

~Code~
101

What prints?

```
x = 4
def thing1():
    x = 20
    return x

def thing2():
    x = -20
    return x

if x > 0:
    winner = thing1
else:
    winner = thing2
print(winner)
```

clicker.cs.illinois.edu

Q3

~Code~
101

winner → thing1()

What prints?

```
def thing1():
    x = 20
    return x

def thing2(var):
    x = -20 + var
    return x

winner = thing2
print(winner(thing1()))
```

20

20

0

clicker.cs.illinois.edu

Q4

~Code~
101

What prints?

```
glob_x = 2

def somethingelse():
    global glob_x
    print(glob_x)
    glob_x += 1

def thing(x):
    x()
    x()

def something(func, out):
    print(out)
    thing(func)
    print(out)

something(somethingelse, "a")
```

somethingelse

'a'

a 0 1 a

clicker.cs.illinois.edu

Q5

~Code~
101

Function Demo

```
1   def thing1(var):
2       print("running thing1 with " + var)
3       def inner(something):
4           print("running inner with " + var + " and " + something)
5       return inner
```

Async

fancy function

Python aysnc - let’s a program switch between coroutines while one is waiting for something slow (waiting for messages over the internet)

Async

```python
import asyncio

async def task1():
    print("task1 start")
    await asyncio.sleep(1)
    print("task1 done")

async def task2():
    print("task2 start")
    await asyncio.sleep(2)
    print("task2 done")

async def main():
    await asyncio.gather(task1(), task2())

asyncio.run(main())
```

```
task1 start
task2 start
task1 done
task2 done
```

Async

Idea - async let's the program switch between coroutines while one is waiting for something slow (waiting for messages over the internet)

- Use the $\boxed{\text{await}}$ keyword to indicate the program can switch if its waiting.

- await needs to be used to invoke another $\boxed{\text{async}}$ function (coroutine)

Layers of a packet

Layer	Protocols	Use
Application	HTTP, SSH, SMTP, etc.	programmer-visible messages
Transport	**TCP, UDP**	reach a specific program
Internet	**IP**, which has two versions: IPv4 and IPv6	reach a specific computer

Async aiohttp lite for processing HTTP

Server wait for connections

↳ coroutine

b'HTTP...'

request

python class object

↳ [Method, path] → func

res = await func(request obj)

res → b'HTTP....' ← response

send that out

Welcome to AIOHTTP

Asynchronous HTTP Client/Server for asyncio and Python.

AIOHTTP

import things... MPB — import web

```
routes = web.RouteTableDef()

@routes.post("/index.htm")
async def get_something(request: web.Request) -> web.Response:
    Get data from request
    ...
    Create and return Response object

if __name__ == '__main__':
    app = web.Application()
    app.add_routes(routes)
    web.run_app(app, ....)
```

Server AIOHTTP Example

```python
from aiohttp.web import RouteTableDef, Application, Request, Response, run_app

routes = RouteTableDef()

@routes.get("/index.html")
async def handle_index(request: Request) -> Response:
    return Response(text=open("index.html").read(), content_type="text/html; charset=utf-8")

if __name__ == '__main__':
    app = Application()
    app.add_routes(routes)
    run_app(app, host='0.0.0.0', port=4582)
```

Client AIOHTTP Example

```html
<!DOCTYPE html>
<html>
<body>
  <button onclick="getData()">GET Request</button>

  <script>
    async function getData() {
      const response = await fetch("/index.html");
      const text = await response.text();
      document.body.innerHTML = text;
    }
  </script>
</body>
</html>
```

get

Server - Client aiohttp

Client

```html
<!DOCTYPE html>
<html>
<body>
  <button onclick="getData()">GET Request</button>

  <script>
    async function getData() {
      const response = await fetch("/index.html");
      const text = await response.text();
      document.body.innerHTML = text;
    }
  </script>
</body>
</html>
```

Request
Get "/index.html"

TCP
handshake

" ..."

server

```python
from aiohttp.web import RouteTableDef, Application, Request, Response, run_app

routes = RouteTableDef()

@routes.get("/index.html")
async def handle_index(request: Request) -> Response:
    return Response(text=open("index.html").read(), content_type="text/html; charset=utf-8")

if __name__ == '__main__':
    app = Application()
    app.add_routes(routes)
    run_app(app, host='0.0.0.0', port=4582)
```

AIOHTTP Reading Request

Request

```
async text(self) -> str
    Return BODY as text using encoding from .charset.
```

```
async json(self, *, loads: Callable[[str], Any] = <function loads at 0x7f8845f885e0>) -> Any
    Return BODY as JSON.
```

```
@routes.get....
async def func(request):
    text = await request.text()
    data = await request.json()
    data["first"]
```

string

python dictionary

AIOHTTP Creating a Response

Response

```
__init__(self, *, body: Any = None, status: int = 200, reason: Optional[str] = None, text: Optional[str] = None, headers: Union[Mapping[Union[str, multidict._multidict.ist
    Initialize self.  See help(type(self)) for accurate signature.
```

```
return Response(text="nooo", status=404)
return Response(text=json.dumps({"name": "hi"}),
                content_type="application/json")
```

Aiohttp

```
from aiohttp import web

routes = web.RouteTableDef()

@routes.post("/echo")
async def echo(request):
    data = await request.text()
    return web.Response(text=data)

app = web.Application()
app.add_routes(routes)

web.run_app(app)
```

Where should the await go?

```
@routes.post("/data")
async def handler(request):
    data = request.text()
    return web.Response(text=data)
```

clicker.cs.illinois.edu

Q6

~Code~
101

Demo

Code is available on campus wire